C04-AT-2(ASSESMENT TOOL 10)
AAMEENA I(192511284)

Question 1: Heap File Organization (Insert, Delete, Sequential Scan)

Python Code

```
"""
1. Heap File Organization
-------------------------
A Heap File stores records in no particular order. New records are
simply appended (fast insert). Deletion is done via "tombstoning"
(marking a record as deleted) rather than physically shifting records,
which is the classic heap-file approach. Sequential scan walks every
slot and skips deleted ones.
"""



class Record:
    def __init__(self, rid, data):
        self.rid = rid          # record id (position in heap)
        self.data = data        # actual record payload (dict)
        self.deleted = False    # tombstone flag

    def __repr__(self):
        status = "DELETED" if self.deleted else "ACTIVE"
        return f"[RID={self.rid} | {self.data} | {status}]"



class HeapFile:
    def __init__(self):
        self.records = []          # the "disk blocks" - simple list
        self.next_rid = 0
        self.free_slots = []       # reuse tombstoned slots on insert

    # ---------- INSERT ----------
    def insert(self, data):
        if self.free_slots:
            rid = self.free_slots.pop()
            self.records[rid] = Record(rid, data)
        else:
            rid = self.next_rid
            self.records.append(Record(rid, data))
            self.next_rid += 1
        print(f"Inserted -> RID {rid}: {data}")
        return rid

    # ---------- DELETE ----------
    def delete(self, rid):
        if 0 <= rid < len(self.records) and not self.records[rid].deleted:
            self.records[rid].deleted = True
            self.free_slots.append(rid)
            print(f"Deleted RID {rid}")
            return True
        print(f"Delete failed: RID {rid} not found or already deleted")
        return False

    # ---------- SEARCH (helper, heap files need full scan) ----------
    def search(self, rid):
        if 0 <= rid < len(self.records) and not self.records[rid].deleted:
            return self.records[rid]
        return None

    # ---------- SEQUENTIAL SCAN ----------
```

```
    def sequential_scan(self):
        print("\n--- Sequential Scan (active records) ---")
        found_any = False
        for rec in self.records:
            if not rec.deleted:
                print(rec)
                found_any = True
        if not found_any:
            print("(heap file is empty)")
        print("----------------------------------------\n")


if __name__ == "__main__":
    heap = HeapFile()

    r1 = heap.insert({"id": 1, "name": "Alice"})
    r2 = heap.insert({"id": 2, "name": "Bob"})
    r3 = heap.insert({"id": 3, "name": "Charlie"})
    r4 = heap.insert({"id": 4, "name": "David"})

    heap.sequential_scan()

    heap.delete(r2)  # delete Bob
    heap.sequential_scan()

    # insert reuses the freed slot from Bob's tombstone
    heap.insert({"id": 5, "name": "Eve"})
    heap.sequential_scan()

    print("Search RID 3:", heap.search(3))
    print("Search RID 1 (after Eve reused a slot, not 1):", heap.search(1))
```

Program Output

```
Inserted -> RID 0: {'id': 1, 'name': 'Alice'}
Inserted -> RID 1: {'id': 2, 'name': 'Bob'}
Inserted -> RID 2: {'id': 3, 'name': 'Charlie'}
Inserted -> RID 3: {'id': 4, 'name': 'David'}

--- Sequential Scan (active records) ---
[RID=0 | {'id': 1, 'name': 'Alice'} | ACTIVE]
[RID=1 | {'id': 2, 'name': 'Bob'} | ACTIVE]
[RID=2 | {'id': 3, 'name': 'Charlie'} | ACTIVE]
[RID=3 | {'id': 4, 'name': 'David'} | ACTIVE]
----------------------------------------

Deleted RID 1

--- Sequential Scan (active records) ---
[RID=0 | {'id': 1, 'name': 'Alice'} | ACTIVE]
[RID=2 | {'id': 3, 'name': 'Charlie'} | ACTIVE]
[RID=3 | {'id': 4, 'name': 'David'} | ACTIVE]
----------------------------------------

Inserted -> RID 1: {'id': 5, 'name': 'Eve'}

--- Sequential Scan (active records) ---
[RID=0 | {'id': 1, 'name': 'Alice'} | ACTIVE]
[RID=1 | {'id': 5, 'name': 'Eve'} | ACTIVE]
[RID=2 | {'id': 3, 'name': 'Charlie'} | ACTIVE]
[RID=3 | {'id': 4, 'name': 'David'} | ACTIVE]
----------------------------------------

Search RID 3: [RID=3 | {'id': 4, 'name': 'David'} | ACTIVE]
```

```
Search RID 1 (after Eve reused a slot, not 1): [RID=1 | {'id': 5, 'name': 'Eve'} | ACTIVE]
```

Question 2: Static Hashing for Student Database (500 records, Chaining)

Python Code

```
"""
2. Static Hashing for a Student Database (500 records)
------------------------------------------------------
A fixed number of buckets (does NOT grow, hence "static"). Each bucket
is a chain (Python list) so multiple keys hashing to the same bucket
are simply appended -> collision resolution by CHAINING.
"""

import random



class Student:
    def __init__(self, roll_no, name, marks):
        self.roll_no = roll_no
        self.name = name
        self.marks = marks

    def __repr__(self):
        return f"(Roll={self.roll_no}, Name={self.name}, Marks={self.marks})"



class StaticHashTable:
    def __init__(self, num_buckets):
        self.num_buckets = num_buckets
        self.buckets = [[] for _ in range(num_buckets)]
        self.collisions = 0
        self.comparisons_on_insert = 0

    def hash_function(self, key):
        # simple static hash: key mod number-of-buckets
        return key % self.num_buckets

    def insert(self, student):
        idx = self.hash_function(student.roll_no)
        if self.buckets[idx]:              # bucket already occupied -> collision
            self.collisions += 1
        self.buckets[idx].append(student)

    def search(self, roll_no):
        idx = self.hash_function(roll_no)
        probes = 0
        for s in self.buckets[idx]:
            probes += 1
            if s.roll_no == roll_no:
                return s, probes
        return None, probes

    def delete(self, roll_no):
        idx = self.hash_function(roll_no)
        chain = self.buckets[idx]
        for i, s in enumerate(chain):
            if s.roll_no == roll_no:
                del chain[i]
                return True
        return False

    def stats(self):
        lengths = [len(b) for b in self.buckets]
```

```
        used = sum(1 for l in lengths if l > 0)
        print("\n--- Hash Table Statistics ---")
        print(f"Number of buckets      : {self.num_buckets}")
        print(f"Buckets in use          : {used}")
        print(f"Total collisions        : {self.collisions}")
        print(f"Max chain length        : {max(lengths)}")
        print(f"Avg chain length (used) : {sum(lengths)/used:.2f}")
        print("------------------------------\n")



if __name__ == "__main__":
    NUM_RECORDS = 500
    NUM_BUCKETS = 100   # fixed/static size, load factor = 5

    table = StaticHashTable(NUM_BUCKETS)

    random.seed(42)
    first_names = ["Aarav", "Vihaan", "Ishaan", "Riya", "Ananya", "Kabir",
                   "Meera", "Diya", "Aditya", "Sara"]

    for roll_no in range(1, NUM_RECORDS + 1):
        name = random.choice(first_names) + str(roll_no)
        marks = random.randint(35, 100)
        table.insert(Student(roll_no, name, marks))

    print(f"Inserted {NUM_RECORDS} student records into {NUM_BUCKETS} buckets.\n")
    table.stats()

    # demo search
    for roll in [1, 250, 499, 999]:
        result, probes = table.search(roll)
        print(f"Search roll_no={roll}: {result}  (probes={probes})")

    # demo delete
    print("\nDeleting roll_no=250 ->", table.delete(250))
    result, probes = table.search(250)
    print(f"Search roll_no=250 after delete: {result}")

    # show one bucket's chain to illustrate collisions
    demo_bucket = table.hash_function(1)
    print(f"\nBucket {demo_bucket} chain (roll numbers that collide there):")
    print([s.roll_no for s in table.buckets[demo_bucket]])
```

Program Output

```
Inserted 500 student records into 100 buckets.



--- Hash Table Statistics ---
Number of buckets      : 100
Buckets in use          : 100
Total collisions        : 400
Max chain length        : 5
Avg chain length (used) : 5.00
------------------------------

Search roll_no=1: (Roll=1, Name=Vihaan1, Marks=38)  (probes=1)
Search roll_no=250: (Roll=250, Name=Aditya250, Marks=75)  (probes=3)
Search roll_no=499: (Roll=499, Name=Sara499, Marks=65)  (probes=5)
Search roll_no=999: None  (probes=5)

Deleting roll_no=250 -> True
Search roll_no=250 after delete: None
```

```
Bucket 1 chain (roll numbers that collide there):
[1, 101, 201, 301, 401]
```

Question 3: Sorted File Organization with Binary Search

Python Code

```
"""
3. Sorted File Organization + Binary Search
-------------------------------------------
Records are physically maintained in ascending order of the primary
key. Insertion/deletion require shifting records to preserve order
(costly, O(n)), but retrieval by primary key is fast: O(log n) via
binary search, unlike the O(n) scan needed on a heap file.
"""



class Record:
    def __init__(self, key, data):
        self.key = key
        self.data = data

    def __repr__(self):
        return f"(Key={self.key}, {self.data})"



class SortedFile:
    def __init__(self):
        self.records = []   # kept sorted by key at all times

    # ---------- INSERT (maintains sort order) ----------
    def insert(self, key, data):
        pos = self._find_insert_position(key)
        if pos < len(self.records) and self.records[pos].key == key:
            print(f"Insert failed: key {key} already exists")
            return False
        self.records.insert(pos, Record(key, data))   # shifts records -> O(n)
        print(f"Inserted key {key} at position {pos}")
        return True

    def _find_insert_position(self, key):
        lo, hi = 0, len(self.records)
        while lo < hi:
            mid = (lo + hi) // 2
            if self.records[mid].key < key:
                lo = mid + 1
            else:
                hi = mid
        return lo

    # ---------- DELETE ----------
    def delete(self, key):
        idx, comparisons = self.binary_search(key)
        if idx == -1:
            print(f"Delete failed: key {key} not found")
            return False
        del self.records[idx]     # shifts records -> O(n)
        print(f"Deleted key {key}")
        return True

    # ---------- BINARY SEARCH ----------
    def binary_search(self, key):
        lo, hi = 0, len(self.records) - 1
        comparisons = 0
        while lo <= hi:
```

```
            mid = (lo + hi) // 2
            comparisons += 1
            if self.records[mid].key == key:
                return mid, comparisons
            elif self.records[mid].key < key:
                lo = mid + 1
            else:
                hi = mid - 1
        return -1, comparisons

    def search(self, key):
        idx, comparisons = self.binary_search(key)
        if idx == -1:
            print(f"Key {key} NOT found (comparisons={comparisons})")
            return None
        print(f"Key {key} found -> {self.records[idx]} (comparisons={comparisons})")
        return self.records[idx]

    def display(self):
        print("\n--- Sorted File Contents ---")
        print(self.records)
        print("-----------------------------\n")


if __name__ == "__main__":
    sf = SortedFile()

    for key, name in [(50, "Eve"), (20, "Bob"), (80, "Grace"), (10, "Alice"),
                      (65, "Frank"), (30, "Charlie"), (40, "David")]:
        sf.insert(key, {"name": name})

    sf.display()

    sf.search(40)
    sf.search(65)
    sf.search(99)   # not present

    sf.delete(20)
    sf.display()

    sf.search(20)   # should now fail
```

Program Output

```
Inserted key 50 at position 0
Inserted key 20 at position 0
Inserted key 80 at position 2
Inserted key 10 at position 0
Inserted key 65 at position 3
Inserted key 30 at position 2
Inserted key 40 at position 3

--- Sorted File Contents ---
[(Key=10, {'name': 'Alice'}), (Key=20, {'name': 'Bob'}), (Key=30, {'name': 'Charlie'}), (Key=40,
{'name': 'David'}), (Key=50, {'name': 'Eve'}), (Key=65, {'name': 'Frank'}), (Key=80, {'name':
'Grace'})]
-----------------------------

Key 40 found -> (Key=40, {'name': 'David'}) (comparisons=1)
Key 65 found -> (Key=65, {'name': 'Frank'}) (comparisons=2)
Key 99 NOT found (comparisons=3)
Deleted key 20

--- Sorted File Contents ---
```

```
[(Key=10, {'name': 'Alice'}), (Key=30, {'name': 'Charlie'}), (Key=40, {'name': 'David'}), (Key=50,
{'name': 'Eve'}), (Key=65, {'name': 'Frank'}), (Key=80, {'name': 'Grace'})]
----------------------------

Key 20 NOT found (comparisons=3)
```

Question 4: B-Tree of Order 4 (Insert, Search, Display)

Python Code

```
"""
4. B-Tree of Order 4
------------------------
Order (m) = 4  ->  each node holds at most m-1 = 3 keys and at most
m = 4 children. A node overflowing (4 keys) is split, and the middle
key is pushed up to the parent - the classic B-Tree insertion
algorithm.

Insert sequence required by the problem: 10, 20, 5, 6, 12, 30, 7, 17
"""



class BTreeNode:
    def __init__(self, leaf=True):
        self.keys = []
        self.children = []
        self.leaf = leaf



class BTree:
    def __init__(self, order=4):
        self.order = order              # max children
        self.max_keys = order - 1       # max keys per node
        self.root = BTreeNode(leaf=True)

    # ---------------- SEARCH ----------------
    def search(self, key, node=None):
        if node is None:
            node = self.root
        i = 0
        while i < len(node.keys) and key > node.keys[i]:
            i += 1
        if i < len(node.keys) and node.keys[i] == key:
            return True
        if node.leaf:
            return False
        return self.search(key, node.children[i])

    # ---------------- INSERT ----------------
    def insert(self, key):
        root = self.root
        if len(root.keys) == self.max_keys:
            new_root = BTreeNode(leaf=False)
            new_root.children.append(root)
            self._split_child(new_root, 0)
            self.root = new_root
            self._insert_non_full(new_root, key)
        else:
            self._insert_non_full(root, key)

    def _insert_non_full(self, node, key):
        if node.leaf:
            i = len(node.keys) - 1
            node.keys.append(None)
            while i >= 0 and key < node.keys[i]:
```

```
                node.keys[i + 1] = node.keys[i]
                i -= 1
            node.keys[i + 1] = key
        else:
            i = len(node.keys) - 1
            while i >= 0 and key < node.keys[i]:
                i -= 1
            i += 1
            if len(node.children[i].keys) == self.max_keys:
                self._split_child(node, i)
                if key > node.keys[i]:
                    i += 1
            self._insert_non_full(node.children[i], key)

    def _split_child(self, parent, i):
        order = self.order
        full_child = parent.children[i]
        mid = (self.max_keys) // 2   # index of the middle key to push up

        new_node = BTreeNode(leaf=full_child.leaf)
        mid_key = full_child.keys[mid]

        new_node.keys = full_child.keys[mid + 1:]
        full_child.keys = full_child.keys[:mid]

        if not full_child.leaf:
            new_node.children = full_child.children[mid + 1:]
            full_child.children = full_child.children[:mid + 1]

        parent.children.insert(i + 1, new_node)
        parent.keys.insert(i, mid_key)

    # ---------------- DISPLAY ----------------
    def display(self, node=None, level=0):
        if node is None:
            node = self.root
            print("\n--- B-Tree Structure (level order) ---")
        print("  " * level + f"Level {level}: {node.keys}")
        for child in node.children:
            self.display(child, level + 1)
        if level == 0:
            print("----------------------------------------\n")



if __name__ == "__main__":
    btree = BTree(order=4)

    keys_to_insert = [10, 20, 5, 6, 12, 30, 7, 17]
    for k in keys_to_insert:
        btree.insert(k)
        print(f"Inserted {k}")

    btree.display()

    for key in [6, 12, 25, 30]:
        found = btree.search(key)
        print(f"Search {key}: {'FOUND' if found else 'NOT FOUND'}")
```

Program Output

```
Inserted 10
Inserted 20
Inserted 5
Inserted 6
Inserted 12
```

```
Inserted 30
Inserted 7
Inserted 17

--- B-Tree Structure (level order) ---
Level 0: [10, 20]
  Level 1: [5, 6, 7]
  Level 1: [12, 17]
  Level 1: [30]
----------------------------------------

Search 6: FOUND
Search 12: FOUND
Search 25: NOT FOUND
Search 30: FOUND
```

Question 5: B+ Tree with Leaf-Level Linking for Range Queries

Python Code

```
"""
5. B+ Tree with Leaf-Level Linking
-----------------------------------
Unlike a plain B-Tree, in a B+ Tree ALL keys/data live in the leaf
nodes; internal nodes only hold routing keys. Leaves are additionally
linked together (leaf.next) to form a singly linked list, which lets
range queries be answered by one descent to the starting leaf followed
by a fast horizontal scan - no need to revisit internal nodes.
"""



class BPlusNode:
    def __init__(self, leaf=True):
        self.leaf = leaf
        self.keys = []
        self.children = []   # internal: child nodes | leaf: unused
        self.values = []     # leaf only: data for each key
        self.next = None     # leaf only: link to next leaf (range queries)



class BPlusTree:
    def __init__(self, order=4):
        self.order = order                  # max children of internal node
        self.max_keys = order - 1
        self.root = BPlusNode(leaf=True)

    # ---------------- SEARCH ----------------
    def search(self, key):
        node = self.root
        while not node.leaf:
            i = 0
            while i < len(node.keys) and key >= node.keys[i]:
                i += 1
            node = node.children[i]
        for i, k in enumerate(node.keys):
            if k == key:
                return node.values[i]
        return None

    # ---------------- RANGE QUERY (uses leaf links) ----------------
    def range_query(self, low, high):
        node = self.root
        while not node.leaf:                # descend to the first relevant leaf
            i = 0
            while i < len(node.keys) and low >= node.keys[i]:
```

```
                i += 1
            node = node.children[i]

        results = []
        while node is not None:             # walk the linked leaves
            for i, k in enumerate(node.keys):
                if low <= k <= high:
                    results.append((k, node.values[i]))
                elif k > high:
                    return results
            node = node.next
        return results

    # ---------------- INSERT ----------------
    def insert(self, key, value):
        root = self.root
        if len(root.keys) == self.max_keys:
            new_root = BPlusNode(leaf=False)
            new_root.children.append(root)
            self._split_child(new_root, 0)
            self.root = new_root
        self._insert_non_full(self.root, key, value)

    def _insert_non_full(self, node, key, value):
        if node.leaf:
            i = 0
            while i < len(node.keys) and node.keys[i] < key:
                i += 1
            node.keys.insert(i, key)
            node.values.insert(i, value)
        else:
            i = 0
            while i < len(node.keys) and key >= node.keys[i]:
                i += 1
            child = node.children[i]
            if len(child.keys) == self.max_keys:
                self._split_child(node, i)
                if key >= node.keys[i]:
                    i += 1
            self._insert_non_full(node.children[i], key, value)

    def _split_child(self, parent, i):
        child = parent.children[i]
        mid = len(child.keys) // 2

        new_node = BPlusNode(leaf=child.leaf)

        if child.leaf:
            # leaf split: copy up the first key of the right half,
            # keep it in both leaves (B+ tree convention), link leaves
            new_node.keys = child.keys[mid:]
            new_node.values = child.values[mid:]
            child.keys = child.keys[:mid]
            child.values = child.values[:mid]

            new_node.next = child.next
            child.next = new_node

            up_key = new_node.keys[0]
        else:
            # internal split: middle key moves UP and is removed from children
            up_key = child.keys[mid]
            new_node.keys = child.keys[mid + 1:]
            new_node.children = child.children[mid + 1:]
            child.keys = child.keys[:mid]
            child.children = child.children[:mid + 1]
```

```
        parent.keys.insert(i, up_key)
        parent.children.insert(i + 1, new_node)

    # ---------------- DISPLAY ----------------
    def display(self, node=None, level=0):
        if node is None:
            node = self.root
            print("\n--- B+ Tree Structure ---")
        prefix = "  " * level + ("Leaf " if node.leaf else "Internal ")
        print(prefix + f"Level {level}: {node.keys}")
        if not node.leaf:
            for child in node.children:
                self.display(child, level + 1)
        if level == 0:
            self._show_leaf_chain()
            print("--------------------------\n")

    def _show_leaf_chain(self):
        node = self.root
        while not node.leaf:
            node = node.children[0]
        chain = []
        while node is not None:
            chain.append(node.keys)
            node = node.next
        print("Leaf chain (linked list) :", " -> ".join(str(c) for c in chain))


if __name__ == "__main__":
    bpt = BPlusTree(order=4)

    for k in [10, 20, 5, 6, 12, 30, 7, 17, 3, 25, 45, 15, 40]:
        bpt.insert(k, f"data_{k}")

    bpt.display()

    print("Search 17 ->", bpt.search(17))
    print("Search 99 ->", bpt.search(99))

    print("\nRange query [15, 30]:")
    for k, v in bpt.range_query(15, 30):
        print(f"  key={k} value={v}")
```

Program Output

```
--- B+ Tree Structure ---
Internal Level 0: [12]
  Internal Level 1: [6, 10]
    Leaf Level 2: [3, 5]
    Leaf Level 2: [6, 7]
    Leaf Level 2: [10]
  Internal Level 1: [20, 25, 30]
    Leaf Level 2: [12, 15, 17]
    Leaf Level 2: [20]
    Leaf Level 2: [25]
    Leaf Level 2: [30, 40, 45]
Leaf chain (linked list) : [3, 5] -> [6, 7] -> [10] -> [12, 15, 17] -> [20] -> [25] -> [30, 40, 45]
--------------------------

Search 17 -> data_17
Search 99 -> None

Range query [15, 30]:
  key=15 value=data_15
```

```
  key=17 value=data_17
  key=20 value=data_20
  key=25 value=data_25
  key=30 value=data_30
```

Question 6: Dense Primary Index on a Sorted File

Python Code

```
"""
6. Dense Primary Index on a Sorted File
----------------------------------------
A DENSE index has one index entry for EVERY record in the (sorted)
data file, each entry storing <key, pointer-to-record>. This gives
very fast lookups (binary search the small index, then a direct jump
to the data record) at the cost of one index entry per record.
"""



class DataRecord:
    def __init__(self, key, data):
        self.key = key
        self.data = data

    def __repr__(self):
        return f"(Key={self.key}, {self.data})"



class IndexEntry:
    def __init__(self, key, pointer):
        self.key = key          # search key
        self.pointer = pointer  # index/offset into the data file

    def __repr__(self):
        return f"<{self.key} -> block {self.pointer}>"



class DensePrimaryIndex:
    def __init__(self):
        self.data_file = []   # sorted list of DataRecord
        self.index = []       # dense index: one entry per data record

    def build(self, records):
        # records: list of (key, data) -- must be primary-key sorted
        self.data_file = [DataRecord(k, d) for k, d in sorted(records, key=lambda r: r[0])]
        self.index = [IndexEntry(rec.key, pos) for pos, rec in enumerate(self.data_file)]
        print(f"Built dense index with {len(self.index)} entries "
              f"(1 index entry per data record).")

    def display_index(self):
        print("\n--- Dense Index Table ---")
        print("KEY".ljust(10) + "POINTER (block/offset)")
        for entry in self.index:
            print(str(entry.key).ljust(10) + str(entry.pointer))
        print("-------------------------\n")

    def search(self, key):
        # binary search directly on the index (index is dense & sorted)
        lo, hi = 0, len(self.index) - 1
        comparisons = 0
        while lo <= hi:
            mid = (lo + hi) // 2
            comparisons += 1
            if self.index[mid].key == key:
```

```
                pointer = self.index[mid].pointer
                record = self.data_file[pointer]
                print(f"Key {key} found via index -> {record} "
                      f"(index comparisons={comparisons})")
                return record
            elif self.index[mid].key < key:
                lo = mid + 1
            else:
                hi = mid - 1
        print(f"Key {key} NOT found (index comparisons={comparisons})")
        return None


if __name__ == "__main__":
    dpi = DensePrimaryIndex()

    records = [
        (101, {"name": "Alice"}), (105, {"name": "Bob"}),
        (110, {"name": "Charlie"}), (115, {"name": "David"}),
        (120, {"name": "Eve"}), (125, {"name": "Frank"}),
        (130, {"name": "Grace"}), (135, {"name": "Heidi"}),
    ]
    dpi.build(records)
    dpi.display_index()

    dpi.search(115)
    dpi.search(130)
    dpi.search(999)   # not present
```

Program Output

```
Built dense index with 8 entries (1 index entry per data record).

--- Dense Index Table ---
KEY       POINTER (block/offset)
101       0
105       1
110       2
115       3
120       4
125       5
130       6
135       7
--------------------------

Key 115 found via index -> (Key=115, {'name': 'David'}) (index comparisons=1)
Key 130 found via index -> (Key=130, {'name': 'Grace'}) (index comparisons=3)
Key 999 NOT found (index comparisons=4)
```

Question 7: Extendible Hashing with Directory Doubling

Python Code

```
"""
7. Extendible Hashing
--------------------------
A directory of 2^global_depth pointers maps to buckets. Each bucket
has its own local_depth <= global_depth. When a bucket overflows:
  - if local_depth == global_depth: the DIRECTORY DOUBLES (global_depth++)
  - the overflowing bucket is split into two, local_depth++ for both
  - directory pointers are re-pointed accordingly
We insert 8 records to demonstrate at least one directory doubling.
"""
```

```
BUCKET_CAPACITY = 2   # small capacity so we see splits/doubling quickly



def hash_function(key):
    # produce a reasonably-mixed binary hash of the key
    return bin(hash(key) & 0xFFFF)[2:].zfill(16)



class Bucket:
    def __init__(self, local_depth):
        self.local_depth = local_depth
        self.records = []   # list of (key, value)

    def is_full(self):
        return len(self.records) >= BUCKET_CAPACITY



class ExtendibleHashTable:
    def __init__(self):
        self.global_depth = 1
        b0 = Bucket(1)
        b1 = Bucket(1)
        self.directory = [b0, b1]

    def _dir_index(self, key):
        h = hash_function(key)
        suffix = h[-self.global_depth:]
        return int(suffix, 2)

    def insert(self, key, value):
        idx = self._dir_index(key)
        bucket = self.directory[idx]

        if not bucket.is_full():
            bucket.records.append((key, value))
            print(f"Inserted {key} -> directory[{idx}] (no split needed)")
            return

        # bucket is full -> need to split
        if bucket.local_depth == self.global_depth:
            self._double_directory()
            idx = self._dir_index(key)
            bucket = self.directory[idx]

        self._split_bucket(bucket)
        # retry insert after split (directory now points correctly)
        self.insert(key, value)

    def _double_directory(self):
        old_size = len(self.directory)
        self.directory = self.directory + self.directory  # duplicate pointers
        self.global_depth += 1
        print(f"*** DIRECTORY DOUBLED *** global_depth is now {self.global_depth} "
              f"(directory size {old_size} -> {len(self.directory)})")

    def _split_bucket(self, old_bucket):
        new_local_depth = old_bucket.local_depth + 1
        new_bucket = Bucket(new_local_depth)
        old_bucket.local_depth = new_local_depth

        # re-distribute directory pointers that pointed to old_bucket
        for i in range(len(self.directory)):
            if self.directory[i] is old_bucket:
```

```
                suffix = bin(i)[2:].zfill(self.global_depth)[-new_local_depth:]
                # the bit that distinguishes old vs new bucket is the
                # newest (most significant of the new_local_depth) bit
                if suffix[0] == '1':
                    self.directory[i] = new_bucket

        # redistribute the old bucket's records between old and new bucket
        old_records = old_bucket.records
        old_bucket.records = []
        new_bucket.records = []
        for key, value in old_records:
            idx = self._dir_index(key)
            self.directory[idx].records.append((key, value))

        print(f"Bucket split -> local_depth now {new_local_depth} for both halves")

    def display(self):
        print(f"\n--- Extendible Hash Table (global_depth={self.global_depth}) ---")
        seen = {}
        for i, b in enumerate(self.directory):
            seen.setdefault(id(b), []).append(i)
        for bucket_id, indices in seen.items():
            bucket = self.directory[indices[0]]
            print(f"Directory{indices} -> Bucket(local_depth={bucket.local_depth}) "
                  f"records={bucket.records}")
        print("-----------------------------------------------------------\n")



if __name__ == "__main__":
    eht = ExtendibleHashTable()

    student_records = [
        (23, "Alice"), (17, "Bob"), (5, "Charlie"), (41, "David"),
        (9, "Eve"), (30, "Frank"), (12, "Grace"), (26, "Heidi"),
    ]

    for key, name in student_records:
        eht.insert(key, name)
        eht.display()
```

Program Output

```
Inserted 23 -> directory[1] (no split needed)

--- Extendible Hash Table (global_depth=1) ---
Directory[0] -> Bucket(local_depth=1) records=[]
Directory[1] -> Bucket(local_depth=1) records=[(23, 'Alice')]
-----------------------------------------------------------

Inserted 17 -> directory[1] (no split needed)

--- Extendible Hash Table (global_depth=1) ---
Directory[0] -> Bucket(local_depth=1) records=[]
Directory[1] -> Bucket(local_depth=1) records=[(23, 'Alice'), (17, 'Bob')]
-----------------------------------------------------------

*** DIRECTORY DOUBLED *** global_depth is now 2 (directory size 2 -> 4)
Bucket split -> local_depth now 2 for both halves
Inserted 5 -> directory[1] (no split needed)

--- Extendible Hash Table (global_depth=2) ---
Directory[0, 2] -> Bucket(local_depth=1) records=[]
Directory[1] -> Bucket(local_depth=2) records=[(17, 'Bob'), (5, 'Charlie')]
Directory[3] -> Bucket(local_depth=2) records=[(23, 'Alice')]
-----------------------------------------------------------
```

```
*** DIRECTORY DOUBLED *** global_depth is now 3 (directory size 4 -> 8)
Bucket split -> local_depth now 3 for both halves
Inserted 41 -> directory[1] (no split needed)

--- Extendible Hash Table (global_depth=3) ---
Directory[0, 2, 4, 6] -> Bucket(local_depth=1) records=[]
Directory[1] -> Bucket(local_depth=3) records=[(17, 'Bob'), (41, 'David')]
Directory[3, 7] -> Bucket(local_depth=2) records=[(23, 'Alice')]
Directory[5] -> Bucket(local_depth=3) records=[(5, 'Charlie')]
----------------------------------------------------------

*** DIRECTORY DOUBLED *** global_depth is now 4 (directory size 8 -> 16)
Bucket split -> local_depth now 4 for both halves
Inserted 9 -> directory[9] (no split needed)

--- Extendible Hash Table (global_depth=4) ---
Directory[0, 2, 4, 6, 8, 10, 12, 14] -> Bucket(local_depth=1) records=[]
Directory[1] -> Bucket(local_depth=4) records=[(17, 'Bob')]
Directory[3, 7, 11, 15] -> Bucket(local_depth=2) records=[(23, 'Alice')]
Directory[5, 13] -> Bucket(local_depth=3) records=[(5, 'Charlie')]
Directory[9] -> Bucket(local_depth=4) records=[(41, 'David'), (9, 'Eve')]
----------------------------------------------------------

Inserted 30 -> directory[14] (no split needed)

--- Extendible Hash Table (global_depth=4) ---
Directory[0, 2, 4, 6, 8, 10, 12, 14] -> Bucket(local_depth=1) records=[(30, 'Frank')]
Directory[1] -> Bucket(local_depth=4) records=[(17, 'Bob')]
Directory[3, 7, 11, 15] -> Bucket(local_depth=2) records=[(23, 'Alice')]
Directory[5, 13] -> Bucket(local_depth=3) records=[(5, 'Charlie')]
Directory[9] -> Bucket(local_depth=4) records=[(41, 'David'), (9, 'Eve')]
----------------------------------------------------------

Inserted 12 -> directory[12] (no split needed)

--- Extendible Hash Table (global_depth=4) ---
Directory[0, 2, 4, 6, 8, 10, 12, 14] -> Bucket(local_depth=1) records=[(30, 'Frank'), (12, 'Grace')]
Directory[1] -> Bucket(local_depth=4) records=[(17, 'Bob')]
Directory[3, 7, 11, 15] -> Bucket(local_depth=2) records=[(23, 'Alice')]
Directory[5, 13] -> Bucket(local_depth=3) records=[(5, 'Charlie')]
Directory[9] -> Bucket(local_depth=4) records=[(41, 'David'), (9, 'Eve')]
----------------------------------------------------------

Bucket split -> local_depth now 2 for both halves
Inserted 26 -> directory[10] (no split needed)

--- Extendible Hash Table (global_depth=4) ---
Directory[0, 4, 8, 12] -> Bucket(local_depth=2) records=[(12, 'Grace')]
Directory[1] -> Bucket(local_depth=4) records=[(17, 'Bob')]
Directory[2, 6, 10, 14] -> Bucket(local_depth=2) records=[(30, 'Frank'), (26, 'Heidi')]
Directory[3, 7, 11, 15] -> Bucket(local_depth=2) records=[(23, 'Alice')]
Directory[5, 13] -> Bucket(local_depth=3) records=[(5, 'Charlie')]
Directory[9] -> Bucket(local_depth=4) records=[(41, 'David'), (9, 'Eve')]
----------------------------------------------------------
```

Question 8: Secondary Storage Block Access Simulation

Python Code

```
"""
8. Secondary Storage Block Access Simulation
------------------------------------------------
Simulates a file stored as fixed-size disk blocks and counts the
number of BLOCK (I/O) ACCESSES needed to retrieve a record by:
```

```
  (a) Sequential search  -> scan blocks one by one until found
  (b) Indexed search     -> use a sparse/dense-style index to jump
                            directly (or near-directly) to the block,
                            modeled as O(log2(num_blocks)) I/Os for a
                            multi-level / B-tree style index, plus 1
                            I/O to fetch the actual data block.
"""

import math



class DiskSimulator:
    def __init__(self, num_records, records_per_block):
        self.num_records = num_records
        self.records_per_block = records_per_block
        self.num_blocks = math.ceil(num_records / records_per_block)
        # records assumed stored in sorted-by-key order across blocks
        print(f"File has {num_records} records, {records_per_block} records/block "
              f"-> {self.num_blocks} disk blocks.")

    def record_block(self, record_position):
        """Which block (0-indexed) holds this record (0-indexed position)?"""
        return record_position // self.records_per_block

    # ---------- SEQUENTIAL SEARCH ----------
    def sequential_search_io(self, record_position):
        target_block = self.record_block(record_position)
        io_count = target_block + 1   # must read every block up to and incl. target
        return io_count

    # ---------- INDEXED SEARCH (B-tree-like multi-level index) ----------
    def indexed_search_io(self, order=100):
        """
        Models a B-tree/B+tree style index over the blocks, where each
        index node (itself one disk block) can hold up to `order`
        pointers/keys. Height of such an index over num_blocks leaf
        entries ~ ceil(log_order(num_blocks)). Add 1 for the final data
        block fetch.
        """
        if self.num_blocks <= 1:
            height = 1
        else:
            height = max(1, math.ceil(math.log(self.num_blocks, order)))
        return height + 1   # index-block reads + 1 data-block read

    def compare(self, record_position, index_order=100):
        seq_io = self.sequential_search_io(record_position)
        idx_io = self.indexed_search_io(index_order)
        print(f"\nSearching for record at position {record_position} "
              f"(block {self.record_block(record_position)}):")
        print(f"  Sequential retrieval I/O operations : {seq_io}")
        print(f"  Indexed retrieval I/O operations    : {idx_io}")
        print(f"  I/O savings using index             : {seq_io - idx_io} "
              f"({100*(seq_io-idx_io)/seq_io:.1f}% fewer block accesses)")



if __name__ == "__main__":
    NUM_RECORDS = 100000
    RECORDS_PER_BLOCK = 50

    disk = DiskSimulator(NUM_RECORDS, RECORDS_PER_BLOCK)

    # Try retrieving records at various positions in the file
    for pos in [0, 500, 25000, 99999]:
        disk.compare(pos, index_order=100)
```

```
    print("\nSummary: sequential I/O grows linearly with record position "
          "(worst case = num_blocks), while indexed I/O stays roughly "
          "constant (logarithmic in the number of blocks), which is why "
          "indexes dramatically reduce disk access for large files.")
```

Program Output

```
File has 100000 records, 50 records/block -> 2000 disk blocks.

Searching for record at position 0 (block 0):
  Sequential retrieval I/O operations : 1
  Indexed retrieval I/O operations    : 3
  I/O savings using index             : -2 (-200.0% fewer block accesses)

Searching for record at position 500 (block 10):
  Sequential retrieval I/O operations : 11
  Indexed retrieval I/O operations    : 3
  I/O savings using index             : 8 (72.7% fewer block accesses)

Searching for record at position 25000 (block 500):
  Sequential retrieval I/O operations : 501
  Indexed retrieval I/O operations    : 3
  I/O savings using index             : 498 (99.4% fewer block accesses)

Searching for record at position 99999 (block 1999):
  Sequential retrieval I/O operations : 2000
  Indexed retrieval I/O operations    : 3
  I/O savings using index             : 1997 (99.8% fewer block accesses)

Summary: sequential I/O grows linearly with record position (worst case = num_blocks), while indexed
I/O stays roughly constant (logarithmic in the number of blocks), which is why indexes dramatically
reduce disk access for large files.
```

Question 9: Sparse Index vs Dense Index (Timing Analysis)

Python Code

```
"""
9. Sparse Index vs Dense Index (with timing analysis)
------------------------------------------------------------
Dense index : one index entry PER RECORD -> O(1) block scan once found.
Sparse index: one index entry PER BLOCK (only for the first/anchor
              record of each block) -> smaller index, but after
              locating the block you must scan within the block.

We build both over the same sorted data file and time many repeated
lookups to compare practical search performance.
"""


import time
import random
import bisect



class SparseVsDenseIndex:
    def __init__(self, num_records, records_per_block):
        self.records_per_block = records_per_block
        # sorted data file: keys 0, 2, 4, 6, ... (sorted, unique)
        self.data = [i * 2 for i in range(num_records)]

        # DENSE index: every key mapped to its position
        self.dense_index = {key: pos for pos, key in enumerate(self.data)}
```

```
        # SPARSE index: only first key of every block
        self.sparse_keys = []     # anchor keys, sorted
        self.sparse_block_start = []  # position in self.data of block start
        for block_start in range(0, num_records, records_per_block):
            self.sparse_keys.append(self.data[block_start])
            self.sparse_block_start.append(block_start)

        print(f"Data file      : {num_records} records")
        print(f"Dense index    : {len(self.dense_index)} entries (1 per record)")
        print(f"Sparse index   : {len(self.sparse_keys)} entries "
              f"(1 per block of {records_per_block} records)")

    # ---------- DENSE SEARCH ----------
    def dense_search(self, key):
        pos = self.dense_index.get(key)
        return pos

    # ---------- SPARSE SEARCH ----------
    def sparse_search(self, key):
        # binary search the sparse index for the right block, then
        # linearly scan within that block (simulates reading the block
        # into memory and scanning it)
        i = bisect.bisect_right(self.sparse_keys, key) - 1
        if i < 0:
            return None
        block_start = self.sparse_block_start[i]
        block_end = min(block_start + self.records_per_block, len(self.data))
        for pos in range(block_start, block_end):
            if self.data[pos] == key:
                return pos
        return None

    def timing_comparison(self, num_lookups=20000):
        random.seed(1)
        lookup_keys = [random.choice(self.data) for _ in range(num_lookups)]

        start = time.perf_counter()
        for k in lookup_keys:
            self.dense_search(k)
        dense_time = time.perf_counter() - start

        start = time.perf_counter()
        for k in lookup_keys:
            self.sparse_search(k)
        sparse_time = time.perf_counter() - start

        print(f"\n--- Timing over {num_lookups} random lookups ---")
        print(f"Dense index search  : {dense_time*1000:.2f} ms total "
              f"({dense_time*1e6/num_lookups:.3f} us/lookup)")
        print(f"Sparse index search : {sparse_time*1000:.2f} ms total "
              f"({sparse_time*1e6/num_lookups:.3f} us/lookup)")
        print(f"Dense index is {sparse_time/dense_time:.2f}x faster in this "
              f"in-memory simulation, but costs {len(self.dense_index)} "
              f"entries vs sparse's {len(self.sparse_keys)} "
              f"({len(self.dense_index)/len(self.sparse_keys):.0f}x more "
              f"index storage).")


if __name__ == "__main__":
    sim = SparseVsDenseIndex(num_records=50000, records_per_block=25)

    print("\nSample lookups:")
    print("dense_search(1000)  ->", sim.dense_search(1000))
    print("sparse_search(1000) ->", sim.sparse_search(1000))
    print("dense_search(99999) ->", sim.dense_search(99999))   # odd -> absent
    print("sparse_search(99999)->", sim.sparse_search(99999))
```

```
    sim.timing_comparison(num_lookups=20000)
```

Program Output

```
Data file      : 50000 records
Dense index    : 50000 entries (1 per record)
Sparse index   : 2000 entries (1 per block of 25 records)

Sample lookups:
dense_search(1000)  -> 500
sparse_search(1000) -> 500
dense_search(99999) -> None
sparse_search(99999)-> None

--- Timing over 20000 random lookups ---
Dense index search  : 3.54 ms total (0.177 us/lookup)
Sparse index search : 19.22 ms total (0.961 us/lookup)
Dense index is 5.43x faster in this in-memory simulation, but costs 50000 entries vs sparse's 2000
(25x more index storage).
```

Question 10: B+ Tree Construction, Traversal, and Range Query [15,45]

Python Code

```
"""
10. B+ Tree Construction, Traversal, and Range Query [15, 45]
-----------------------------------------------------------------
Builds a B+ Tree (order 4), performs an in-order traversal (via the
linked leaves, which is the efficient way for a B+ Tree), and then
executes a range query returning every record with key in [15, 45].
"""



class BPlusNode:
    def __init__(self, leaf=True):
        self.leaf = leaf
        self.keys = []
        self.children = []
        self.values = []
        self.next = None



class BPlusTree:
    def __init__(self, order=4):
        self.order = order
        self.max_keys = order - 1
        self.root = BPlusNode(leaf=True)

    def insert(self, key, value):
        root = self.root
        if len(root.keys) == self.max_keys:
            new_root = BPlusNode(leaf=False)
            new_root.children.append(root)
            self._split_child(new_root, 0)
            self.root = new_root
        self._insert_non_full(self.root, key, value)

    def _insert_non_full(self, node, key, value):
        if node.leaf:
            i = 0
            while i < len(node.keys) and node.keys[i] < key:
                i += 1
            node.keys.insert(i, key)
            node.values.insert(i, value)
```

```
        else:
            i = 0
            while i < len(node.keys) and key >= node.keys[i]:
                i += 1
            child = node.children[i]
            if len(child.keys) == self.max_keys:
                self._split_child(node, i)
                if key >= node.keys[i]:
                    i += 1
            self._insert_non_full(node.children[i], key, value)

    def _split_child(self, parent, i):
        child = parent.children[i]
        mid = len(child.keys) // 2
        new_node = BPlusNode(leaf=child.leaf)

        if child.leaf:
            new_node.keys = child.keys[mid:]
            new_node.values = child.values[mid:]
            child.keys = child.keys[:mid]
            child.values = child.values[:mid]
            new_node.next = child.next
            child.next = new_node
            up_key = new_node.keys[0]
        else:
            up_key = child.keys[mid]
            new_node.keys = child.keys[mid + 1:]
            new_node.children = child.children[mid + 1:]
            child.keys = child.keys[:mid]
            child.children = child.children[:mid + 1]

        parent.keys.insert(i, up_key)
        parent.children.insert(i + 1, new_node)

    # ---------------- TRAVERSAL ----------------
    def display_tree(self, node=None, level=0):
        if node is None:
            node = self.root
            print("\n--- B+ Tree Structure ---")
        tag = "Leaf" if node.leaf else "Internal"
        print("  " * level + f"{tag} (level {level}): {node.keys}")
        if not node.leaf:
            for child in node.children:
                self.display_tree(child, level + 1)
        if level == 0:
            print("--------------------------\n")

    def inorder_via_leaves(self):
        """Efficient in-order traversal: descend once to leftmost leaf,
        then just follow the `next` pointers - no re-visiting internal nodes."""
        node = self.root
        while not node.leaf:
            node = node.children[0]
        result = []
        while node is not None:
            for k, v in zip(node.keys, node.values):
                result.append((k, v))
            node = node.next
        return result

    # ---------------- RANGE QUERY ----------------
    def range_query(self, low, high):
        node = self.root
        while not node.leaf:
            i = 0
            while i < len(node.keys) and low >= node.keys[i]:
                i += 1
            node = node.children[i]
```

```
        results = []
        while node is not None:
            for k, v in zip(node.keys, node.values):
                if low <= k <= high:
                    results.append((k, v))
                elif k > high:
                    return results
            node = node.next
        return results


if __name__ == "__main__":
    bpt = BPlusTree(order=4)

    keys = [10, 25, 5, 40, 15, 45, 20, 8, 35, 50, 12, 30, 22]
    for k in keys:
        bpt.insert(k, f"record_{k}")

    bpt.display_tree()

    print("In-order traversal (via linked leaves):")
    print(bpt.inorder_via_leaves())

    print("\nRange query for keys between 15 and 45:")
    results = bpt.range_query(15, 45)
    for k, v in results:
        print(f"  MATCH -> key={k}, value={v}")
    print(f"\nTotal matching records: {len(results)}")
```

Program Output

```
--- B+ Tree Structure ---
Internal (level 0): [25]
  Internal (level 1): [10, 15]
    Leaf (level 2): [5, 8]
    Leaf (level 2): [10, 12]
    Leaf (level 2): [15, 20, 22]
  Internal (level 1): [40]
    Leaf (level 2): [25, 30, 35]
    Leaf (level 2): [40, 45, 50]
--------------------------

In-order traversal (via linked leaves):
[(5, 'record_5'), (8, 'record_8'), (10, 'record_10'), (12, 'record_12'), (15, 'record_15'), (20,
'record_20'), (22, 'record_22'), (25, 'record_25'), (30, 'record_30'), (35, 'record_35'), (40,
'record_40'), (45, 'record_45'), (50, 'record_50')]

Range query for keys between 15 and 45:
  MATCH -> key=15, value=record_15
  MATCH -> key=20, value=record_20
  MATCH -> key=22, value=record_22
  MATCH -> key=25, value=record_25
  MATCH -> key=30, value=record_30
  MATCH -> key=35, value=record_35
  MATCH -> key=40, value=record_40
  MATCH -> key=45, value=record_45

Total matching records: 8
```